

Assignment 2: Yarrow-Mullein Bank Event-Driven Ingestion Pipeline

Student Information

- **Name:** Alex Wolfe
- **Date completed:** 9/24/2026
- **GitHub repository URL:** <https://github.com/Alex-wolfe-rrc/Cloud-assignment-2>

Architecture Diagram

Your diagram should show **incoming/** as the input prefix, **processed/** as the output prefix, the **ObjectCreated** event, and CloudWatch logging.

Implementation Summary

Briefly explain how your pipeline validates Yarrow-Mullein Bank customer service requests and writes processed output.

The pipeline uses an Amazon S3 bucket to receive customer service request JSON files under the **incoming/** prefix. An S3 **ObjectCreated** event triggers a Python Lambda function, which validates the required fields in each request. Valid requests are written to the **processed/** prefix, while invalid requests are recorded in CloudWatch Logs.

Required Evidence

S3 Bucket and Prefixes

Lambda Function and Permissions

S3 Trigger

Valid Request Processing

Invalid Request Handling

JSON Data Shape

valid-requests.json contains 8 valid request objects. invalid-requests.json contains 2 invalid request objects. Both files use a top-level "requests" array. All requestId values are unique. The invalid records contain either a missing or empty required field.

Negative Trigger Tests

Anything outside of the scope, such as a .txt file or something outside of **incoming/** would not run the lambda function. it only accepts objects whos key starts with **incoming/** and ends with **.json**

Reflection Questions

1. What event causes the Lambda function to run?

An S3 ObjectCreated event for a .json object uploaded under the incoming/ prefix.

2. Why should the function validate a customer service request before creating the processed output?

because you wouldn't want a complete or invalid customer request entering the processed data.

3. What S3 and CloudWatch permissions does the Lambda execution role require, and why?

S3 needs to read incoming files, write processed files, and cloudwatch needs to create logs, and write logs. It needs these permissions to create the right output and log the result.

4. How would you prevent the Lambda function from being triggered by its own output files?

The prefix filter prevents Lambda's output from triggering it again.

5. If you were deploying outside AWS Academy, how would you scope the Lambda permissions to only the **incoming/** and **processed/** prefixes?

I would use incoming/ for reads, and processed/ for writes, rather than granting access to the entire bucket. It has no reason to read any other information, but it needs those specific permissions to process and log the data.